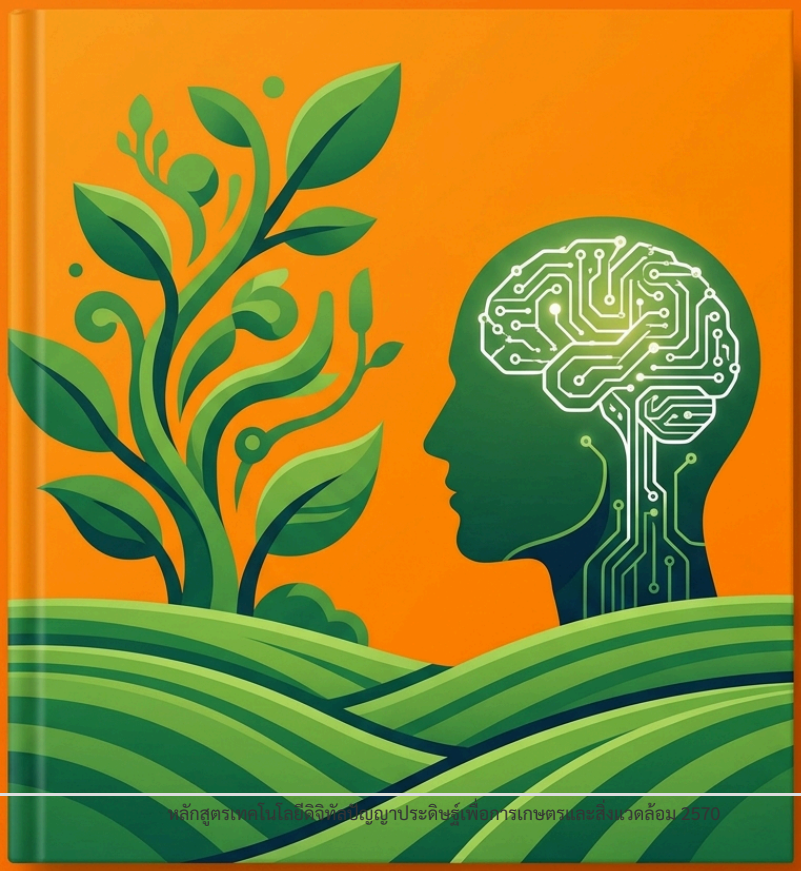
Python xAI

< AI Agent Co-Creator >

Student Survival Lab - Practical AI Engineering



คู่มือนักเรียน: Python x AI 2026 (ฉบับเจาะลึก 3 ระดับ)

Next-Gen Scientist Edition - เส้นทางสู่การเป็นผู้สร้างนวัตกรรม AI เพื่อโลกที่ยั่งยืน

คัดเลือกระดับความท้าทาย (Choose Your Path)

ก่อนเริ่มภารกิจ โปรดพิจารณาระดับความถนัดของคุณ เพื่อเลือกชุดโค้ดและแบบฝึกหัดที่เหมาะสม:

- **Junior (ม.ต้น)**: เน้นตรรกะพื้นฐาน การใช้ตัวแปร และการสื่อสารกับ AI เบื้องต้น
- **Senior (ม.ปลาย)**: เน้นการวิเคราะห์ข้อมูล (ML), การเขียนเงื่อนไขที่ซับซ้อน และผลลัพธ์เชิงสถิติ
- **Vocational/Research (ปวช./วิจัย)**: เจาะลึกการรันโค้ดระดับสูง, Computer Vision (HSV) และระบบอัจฉริยะแบบบูรณาการ

บทที่ 1: ห้องแล็บลอยฟ้าและตรรกะมหัศจรรย์

หัวใจสำคัญ: การใช้ Google Colab และการสั่งงานคอมพิวเตอร์อย่างเป็นระบบ

พื้นฐานการใช้งาน (All Levels)

- `Shift + Enter` : เพื่อ **รันโค้ด** (Run Cell)
- `!pip install ...` : (Vocational) สำหรับการติดตั้งไลบรารีพิเศษใน Terminal

ภารกิจการเขียนโค้ด (Coding Missions)

ระดับ	ภารกิจ (Mission)	ตัวอย่างโค้ด (Real Code)
Junior	**Greeting Bot**: สร้างบอททักทายที่เก็บชื่อเราลงในตัวแปร	`name = input("ชื่ออะไรครับ?")` `print("สวัสดีคุณ " + name)`
Senior	**Data Threshold**: คัดกรองข้อมูลอุณหภูมิที่ "อันตราย"	`if temp > 40:` `print("Warning: Too Hot!")`
Vocational	**System Loop**: วนลูปอ่านค่าจากเซนเซอร์หลายจุด	`for sensor in farm_data:` `process(sensor)`

บทที่ 2: พลังแห่งการพยากรณ์และการอธิบายเหตุผล

หัวใจสำคัญ: สอนให้ AI ทายอนาคต และทำความเข้าใจ "ทำไม AI ถึงคิดแบบนั้น"

ภารกิจ Machine Learning

- **Junior (Conceptual)**: เรียนรู้ว่า AI มองเห็น "รูปแบบ" (Pattern) ในข้อมูลดินได้อย่างไร
- **Senior (Implementation)**:

.py / .ipynb

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
# ดูเหตุผล (xAI)
print(model.feature_importances_)
```

- **Vocational (Optimization)**: การทำ **Data Cleaning** (fillna) ก่อนการฝึกสอน เพื่อให้โมเดลมีความแม่นยำสูงสุด

บทที่ 3: สมองดิจิทัล (Deep Learning & Neural Networks)

หัวใจสำคัญ: การเลียนแบบเซลล์ประสาทมนุษย์

โครงสร้างสมอง AI (Topology Analysis)

1. **Input Layer**: รับภาพใบพืช
2. **Hidden Layer (ReLU)**: กรองข้อมูลที่ "ไม่สำคัญ" ทิ้งไป
3. **Dropout Layer**: (Senior/Voc-Only!) สุ่มลบเซลล์ประสาทบางตัวเพื่อป้องกันการ "จำเพาะเจาะจงเกินไป" (Overfitting)

บทที่ 4: ดวงตาอัจฉริยะและการสื่อสาร (Vision & NLP)

หัวใจสำคัญ: การประมวลผลวิดีโอแบบ Real-time และการสรุปใจความสำคัญ

Computer Vision (OpenCV)

- **Junior**: ทำความเข้าใจค่าการสะท้อนแสง (Pixel Values)
- **Senior/Vocational**: การตัดกรองสี (Masking) ด้วยระบบ **HSV**

.py / .ipynb

```
import cv2
mask = cv2.inRange(hsv, lower_brown, upper_brown)
# เจาะหา "จุดสีน้ำตาล" ของโรคกระบนใบไม
```

AI Liaison (NLP)

- **Mission**: แมปคำพูดของคนให้เป็น "ป้ายคำสั่ง" (**Intents**) เพื่อให้ AI สตาร์ทเครื่องยนต์หรือเปิดน้ำได้อัตโนมัติ

บทที่ 5: Pygame Zero Simulation (The Visualization)

หัวใจสำคัญ: การเปลี่ยนโค้ด AI ให้เป็นภาพเคลื่อนไหวที่จับต้องได้

ภารกิจจำลองโลกเกษตร

- **Junior**: ฝึกขยับตัวละครด้วยลูกศรบนคีย์บอร์ด
- **Senior**: สร้างระบบตรวจจับการชน (Collision) เมื่อรถเจอพืช
- **Vocational**: พัฒนา **Agri-Agent** ที่ขยับหาพืชที่เป็นโรคเองโดยอัตโนมัติ

บทที่ 6: Agri-Hackathon Stage (The Integration)

หัวใจสำคัญ: การนำนวัตกรรมมาแก้ปัญหาพื้นที่จริง

ตารางการทำงานสำหรับนักนวัตกรรม

1. **Define Problem**: เลือกปัญหาจากการเกษตรแถวบ้าน
2. **Select Level Core**: เลือกระดับความยากของ AI ที่จะใช้
3. **Build & Test**: รัน Prototype ใน Colab พร้อมโชว์ผลลัพธ์ผ่านกราฟหรือภาพสแกน
4. **Pitch**: นำเสนอด้วย Template: "ปัญหาคืออะไร -> AI แก้ยังไง -> ผลลัพธ์ที่จึ้งคืออะไร"

พจนานุกรม และ โค้ดอ้างอิง (Quick Reference)

- **ReLU**: ฟังก์ชันกระตุ้นที่จะส่งผ่านเฉพาะค่าที่เป็นบวกเท่านั้น
- **Softmax**: ฟังก์ชันตัดสินในเลเยอร์ท้ายสุด เพื่อบอก "ความน่าจะเป็น" ของแต่ละคำตอบ
- **HSV**: ระบบสีที่แยกเฉดสี (Hue) ความอิ่มตัว (Saturation) และค่าความสว่าง (Value) ออกจากกัน (แม่นยำกว่า RGB ในงานวิจัย)

> *[!IMPORTANT] **หมายเหตุจากพี่ยักษ์**: คู่มือนี้จัดทำขึ้นเพื่อให้ทุกคน 'เริ่มต้น' ได้ ไม่ว่าคุณจะอยู่ระดับไหน... ขอแค่มีความกล้าที่จะลองพิมพ์ Code บรรทัดแรก อนาคตก็เริ่มขึ้นแล้วครับ!*